

6. MVP Design and Build

6.1 Purpose of this stage

This stage turns the product definition into a **real, minimal product**—the first live version that serious shooters can use in their training.

In product development and lean startup practice, an MVP is the simplest version of a product that delivers core value and allows you to learn from real usage, without building everything you eventually envision.

For the shooters pocket app, the MVP must execute the core loop: **plan session** → **train** → **log** → **review progress**, tailored to ISSF pistol/rifle.

6.2 MVP philosophy for this app

We adopt three principles that align with MVP best practices:[]

1. **Core value only**

The MVP focuses on the minimum set of features needed for a shooter to run structured sessions, log data, and see progress—no extras.

2. **Simple, reliable workflows**

It is better to have a few workflows that are smooth and reliable than many that are shallow or buggy.

3. **Designed for learning**

Architecture and instrumentation must allow us to observe usage, test variations, and iterate quickly, consistent with the build–measure–learn loop.[]

6.3 MVP scope: what we actually build

The **MVP scope** for the shooters pocket app is:

1. **Onboarding and shooter profile**

- Simple sign-up (email or phone + password).
- Basic profile: discipline (pistol/rifle), rough training frequency, current goal (e.g., “improve match average,” “build consistency”).

This tailors session suggestions and analytics to their discipline without overcomplicating setup.

2. **Session templates (limited but focused)**
 - A curated set of **5–10 high-quality session templates** covering:
 - Technical drills (hold, trigger control, alignment).
 - Match simulation sessions.
 - Dry-fire sessions.
 - Each template includes: duration, drill sequence, and any notes/instructions that reflect ISSF best practices.
3. **Session execution and logging**
 - Screens to run a session with clear steps, and simple fields to log:
 - Scores/series.
 - Number of shots.
 - Optional notes (conditions, feelings, adjustments).
 - Logging designed to be **fast and low-friction** so shooters can use it in real ranges.
4. **Basic analytics dashboard**
 - A small set of charts:
 - Average training scores over time.
 - Practice volume (sessions per week, shots logged).
 - Simple “best score / recent trend” indicators.
 - These analytics provide immediate feedback and a sense of progress without advanced data science.
5. **Training history view**
 - A timeline of past sessions and key stats, so shooters can scroll through their training story and review past notes and performance.

This aligns with common MVP guidance: build a basic model with only core features that cater to the target audience’s primary needs.

6.4 Out-of-scope in MVP (reaffirmed)

The MVP will intentionally **exclude**:

- Coach dashboards and multi-shooter views (only basic hints may be prepared for later).
- Social features (feeds, comments, likes).
- Hardware integrations (smart targets, wearables, cameras).
- Advanced analytics or AI coaching.
- Multi-sport support.

These features may appear in later versions, but they are not required to validate that the core training loop delivers value.

6.5 Design priorities and UX principles

Design choices must reflect the realities of precision shooting and good UX practice:

1. **Low cognitive load**
 - UI should be clean, minimal, and calm—no bright distractions or clutter.
 - Navigation should center around a few main sections: “Today’s Session,” “Log,” “Progress,” and “History.”
2. **Fast logging flow**
 - Logging must be achievable with one hand and minimal taps, allowing shooters to focus on the target and environment.
 - Default values and smart presets should reduce friction.
3. **Clarity of feedback**
 - Analytics and summaries should show a few key metrics with simple visuals, not dense tables or jargon.
 - The aim is “at a glance” understanding of whether training is trending in the right direction.
4. **Consistency and reliability**
 - No experimental UI patterns that confuse; adopt standard mobile patterns for tabs, lists, and charts.
 - Emphasize reliability over cleverness—shooters must trust it as a daily tool.

These priorities align with product development guidance that stresses usability and clear value delivery in the early stages.[]

6.6 Technical architecture choices (high-level)

The MVP architecture should balance speed, simplicity, and future extensibility:[]

- **Client platform:**
 - Start with mobile (Android first, given India base, or cross-platform framework like React Native/Flutter) to ensure shooters can use it at the range.
 - **Backend:**
 - Lightweight backend handling authentication, session templates, logging, analytics summaries, and data storage.
 - Cloud-hosted (e.g., managed services) to avoid infrastructure overhead.
 - **Data model:**
 - Core entities: User, Session Template, Training Session, Metrics.
 - Design the schema to allow adding more metrics and disciplines later without breaking existing data.
 - **Instrumented analytics:**
 - Event tracking for key actions (session start, session completion, logging events, analytics view opens) to feed product metrics defined in Stage 5.[]
-

6.7 Build plan and roles

Given your constraints, the build plan should be lean and role-efficient:

- **You (founder/product/subject-matter):**
 - Own product specification, session template content, and training logic.
 - Make decisions on what sessions, metrics, and flows reflect real ISSF training.
- **Designer (contract or part-time):**
 - Translate workflows into simple, discipline-appropriate UI.
 - Ensure consistency across screens and states.
- **Developer(s):**
 - Implement the mobile app and backend according to specs.
 - Work in short cycles with frequent checkpoints to keep scope under control.

Product development documentation advises tight collaboration between product, design, and engineering at this stage to maintain alignment with the defined goals and user needs.

6.8 Build–measure–learn baked into MVP

From day one, the MVP must be ready for **build–measure–learn**

- **Build**
 - Release a working version to a controlled cohort of serious shooters, not the general public.
 - **Measure**
 - Track the North Star metric (weekly active training shooters) and supporting metrics (activation, sessions per shooter, retention, time to first insight).
 - **Learn**
 - Use metrics and direct feedback to adjust: remove friction, tune templates, simplify flows, or change analytics emphasis based on what shooters actually use.
-

6.9 Release strategy: controlled launch

Instead of a broad app store launch, early releases should be **controlled and intentional**:

- **Pilot cohorts**
 - Hand-picked shooters and coaches in India and possibly a few in Europe, chosen for seriousness and willingness to provide feedback.
- **Guided onboarding**
 - Personal or small-group onboarding sessions where you explain the app, help set goals, and show how to use sessions and analytics.

- **Feedback channels**
 - Direct communication channels (WhatsApp groups, email, or in-app feedback) where pilots can report bugs, suggest improvements, and share experiences promptly.

This mirrors common “market testing” and “limited rollout” stages in product development, where products are exposed to a small audience to refine before wider release.

6.10 Exit conditions for Stage 6

Stage 6 is complete when:

- A **working MVP** exists that implements the defined scope and core workflows (training sessions, logging, analytics, history).
- The app is instrumented with basic analytics and logging, connected to the success metrics defined in Stage 5.
- A small but real cohort of ISSF shooters is using it in real training contexts, with clear feedback loops in place.

At that point, you move into Stage 7 (Testing, Validation, and Iteration) formally, even though early testing begins as soon as pilots touch the MVP.